Simulation Style Guide

Ivan Jacob Agaloos Pesigan

This guide covers `.sim` in simulation projects. Agents follow `.setup/ai/style/simulation.md`, which follows this guide and is changed together with it. Existing simulations follow it when next reused.

1 Layout

- `README.md`: how to set up, run, and summarize. It also marks the project as a simulation.
- `sim-args.R`: conditions, replications, the failure classes, and the attempts allowed.
- `sim.R` and `sim.sh`: one block of cells, and the SLURM job array.
- `sim-compress.R` and `sim-compress.sh`: collect and compress results.
- `sum-*.R` and `sum-*.sh`: summaries for the paper.
- `monitor.sh`: progress, and the cells to check.

The functions that do the work live in `R`, documented and tested; scripts only call them. Set the thread counts of OpenMP, MKL, and OpenBLAS to one: the job array is the only parallel layer.

2 Seeds

A cell is one `taskid` and `repid`. Its seed comes from those two and the attempt number, so any cell can be rerun alone:

```
seed <- ((attempt - 1L) * tasks + (taskid - 1L)) * reps + repid
```

No two cells or attempts share a seed while `repid` is at most `reps` and `taskid` is at most `tasks`. Keep the product of attempts, tasks, and replications below the largest integer in R.

3 Failures

- Each project lists the failure classes that allow a new seed in `sim-args.R`.
- Anything else is a result, or an error to fix: it never gets a new seed.
- A listed failure moves the cell to the next attempt, up to the maximum in `sim-args.R`.
- A crash (a segfault, or an out-of-memory kill) counts as a failed attempt of the cell that was running.
- Never set a seed by hand for one cell.
- The cell's output records `taskid`, `repid`, `attempt`, the seed, and, for each failed attempt, its seed and failure class. It does not keep a failed attempt's results.
- The summary reports the reseeded cells per condition and the failure rate, with the results: keeping only successful attempts conditions on convergence.

4 Output

- One file per cell, with the condition and replication in its name.
- Resume by skipping files that exist.
- Write to a temporary name and rename, so a killed job leaves no half file.
- Save `.Rds` files with `xz`; every file stays under 50 MB, and only summaries go to Overleaf.
- `tryCatch()` prints the task and replication of a cell to check, so that `monitor.sh` can list them.

5 SLURM

The default is a job array of blocks, with the replications in series inside each block. SLURM does the managing; there is no GNU Parallel and no whole node. This starts in free cores, loads R and the packages once per block, and loses one block when a node fails. It is not one array element per cell: clusters limit the jobs a user can submit and the size of an array.

5.1 Measure, Then Size

1. Measure: run a few cells of the slowest condition, and read `seff` and `sacct` for time and memory.
2. Size: choose replications per block so that a block runs about one to four hours, within the job limit. Write the measured numbers in `.sim/README.md`.
3. Check after the run: read `seff` on some elements, and update the numbers.

5.2 Blocks

An element's index gives its task and range of replications by a fixed formula, kept in `R`, so a block is reproducible and can be rerun alone.

- Run the cells in a loop in one R process, with the seed and attempt rules above for each cell.
- Wrap each cell in `tryCatch()`, clear large objects between cells, and call `gc()`.
- Crashes: `tryCatch()` does not survive a segfault or an out-of-memory kill. Before a cell starts, write a marker with its task, replication, and attempt, and remove it when the cell ends. A rerun that finds a marker counts that attempt as failed and moves on, so a block does not hit the same crash forever.
- Rerun only the missing blocks, with a list in `sbatch --array`.

5.3 Settings

- One core per element, and the measured peak memory plus a margin.
- Time: the measured block time plus a margin, not the partition's maximum. A shorter limit can start sooner, and a block that hits its limit loses only unfinished cells.
- Cap the elements running at once with `%N` in `--array`, starting low on a new simulation.
- Mail: `--mail-type=END,FAIL`, not `ALL`.
- Output and error files per element.
- Keep temporary files on the node, and compress when a stage ends. Summaries and compression also go through `sbatch`.

5.4 When to Use GNU Parallel

Use a whole node with GNU Parallel only when a cell crashes often, so that one process per cell pays for itself, or when a job needs a node's memory. Before moving a real simulation to blocks, run one condition both ways and compare core-hours and time to finish.

6 Running

- Before a full run: one or two replications inside the image, and `sbatch --test-only`.
- Agents never submit the full run: they give the user the command, with the measured time and memory.
- Results reach the paper through `Fig*()` and `.Tabular()`.

References

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. https://www.R-project.org/